Mathematical Foundations, Architecture Diagrams, and State Analysis of Modern LLM Training Methods

Chief Strategist J

June 12, 2026

Contents

Part I
Foundation Training Methods

Pretraining

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \ldots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^{T} P(x_t \mid x_{<t}) \tag{1}$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \tag{2}$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \tag{3}$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \tag{4}$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{5}$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t. The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{6}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{7}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{8}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \tag{9}$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi\right)\right) \tag{10}$$

Architecture Flow Diagram

State Transition Table

Python Implementation

```python
import torch
import torch.nn as nn
import math

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
```

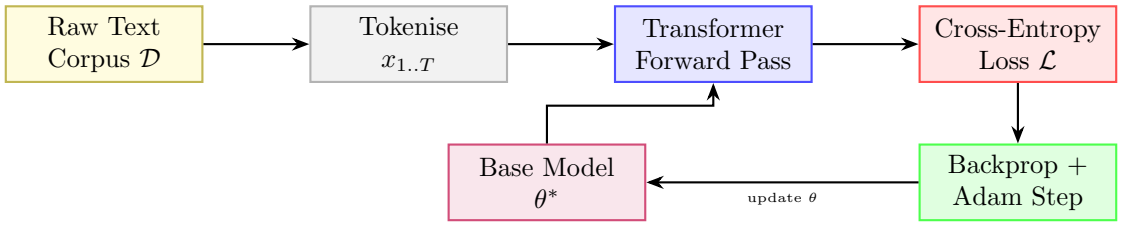

Figure 1: Pretraining pipeline: corpus sampling, forward pass, loss computation, and weight update loop.

Property	**Before**	**After**
Parameter source	Random init $\theta_0 \sim \mathcal{N}(0, \sigma^2)$	Converged weights θ^*
Training corpus	Not consumed	Trillions of tokens processed
Perplexity	Very high ($> 10^3$)	Low (< 20 on held-out data)
World knowledge	None	Broad language & factual knowledge
requires_grad	True (all params)	True (all params)

Table 1: State properties before and after pretraining.

```
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, x):
        # x shape: (batch_size, seq_len, d_model)
        b, t, c = x.size()

        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)

        # Compute scaled query-key dot product attention
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)

        # Apply causal attention mask
        mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
        scores = scores + mask.view(1, 1, t, t)

        attn_weights = torch.softmax(scores, dim=-1)
        out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
        out = out.transpose(1, 2).contiguous().view(b, t, c)
        return self.out_proj(out)

class PretrainingCausalLM(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads):
        super().__init__()
        self.attn = CausalSelfAttention(d_model, num_heads)
        self.vocab_proj = nn.Linear(d_model, vocab_size)
        self.loss_fn = nn.CrossEntropyLoss(reduction='mean')

    def forward(self, x, targets=None):
        # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
        # targets: target token ids, shape: (batch_size, seq_len)
        h = self.attn(x)
        logits = self.vocab_proj(h)

        loss = None
        if targets is not None:
            shift_logits = logits[..., :-1, :].contiguous()
            shift_targets = targets[..., 1:].contiguous()
            loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1)
    )

        return logits, loss
```

Continued Pretraining (CPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \ldots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^{T} P(x_t \mid x_{<t}) \tag{11}$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \tag{12}$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \tag{13}$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \tag{14}$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{15}$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t. The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{16}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{17}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{18}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \tag{19}$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \tag{20}$$

Architecture Flow Diagram

State Transition Table

Property	**Before**	**After**
Starting checkpoint	Pretrained θ_{base}	$\theta' = \theta_{\text{base}} + \Delta\theta$
New domain knowledge	Absent	Incorporated
Original knowledge	Retained	Partially retained via replay
Learning rate schedule	Finished	Warm-restart applied
Perplexity on new domain	High	Significantly reduced

Table 2: State properties before and after continued pretraining.

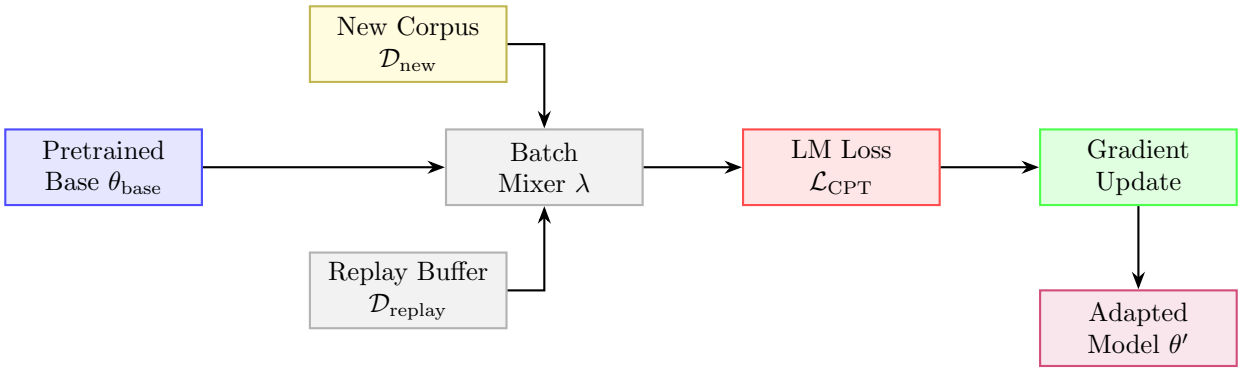

Figure 2: CPT mixes new corpus and replay buffer batches to update a pretrained checkpoint.

Python Implementation

```python
import torch
import torch.nn as nn
import math

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, x):
        # x shape: (batch_size, seq_len, d_model)
        b, t, c = x.size()

        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)

        # Compute scaled query-key dot product attention
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)

        # Apply causal attention mask
        mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
        scores = scores + mask.view(1, 1, t, t)

        attn_weights = torch.softmax(scores, dim=-1)
        out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
        out = out.transpose(1, 2).contiguous().view(b, t, c)
        return self.out_proj(out)

class PretrainingCausalLM(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads):
        super().__init__()
        self.attn = CausalSelfAttention(d_model, num_heads)
        self.vocab_proj = nn.Linear(d_model, vocab_size)
        self.loss_fn = nn.CrossEntropyLoss(reduction='mean')

    def forward(self, x, targets=None):
        # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
        # targets: target token ids, shape: (batch_size, seq_len)
        h = self.attn(x)
        logits = self.vocab_proj(h)

        loss = None
        if targets is not None:
            shift_logits = logits[..., :-1, :].contiguous()
            shift_targets = targets[..., 1:].contiguous()
            loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1)
    )

        return logits, loss
```

Domain Adaptive Pretraining (DAPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \ldots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^{T} P(x_t \mid x_{<t}) \tag{21}$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \tag{22}$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \tag{23}$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \tag{24}$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{25}$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t. The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{26}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{27}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{28}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \tag{29}$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi\right)\right) \tag{30}$$

Architecture Flow Diagram

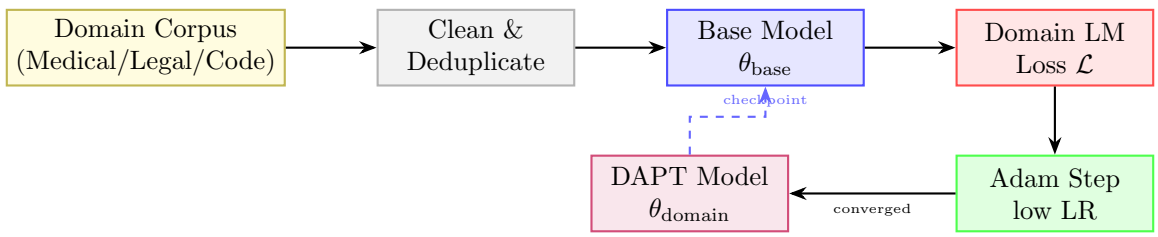

Figure 3: DAPT pipeline: domain corpus is cleaned, fed into the base model, and trained with standard LM loss.

Property	Before	After
Domain vocabulary familiarity	Low	High
Domain perplexity	High ($\rho \gg 1$)	Low (near 1)
General language ability	Strong	Marginally reduced
Downstream task (same domain)	Weak	Significantly stronger
Training data type	General web text	Domain-specific unlabelled

Table 3: State properties before and after domain adaptive pretraining.

State Transition Table

Python Implementation

```python
import torch
import torch.nn as nn
import math

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, x):
        # x shape: (batch_size, seq_len, d_model)
        b, t, c = x.size()

        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)

        # Compute scaled query-key dot product attention
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)

        # Apply causal attention mask
        mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
        scores = scores + mask.view(1, 1, t, t)

        attn_weights = torch.softmax(scores, dim=-1)
        out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
        out = out.transpose(1, 2).contiguous().view(b, t, c)
        return self.out_proj(out)

class PretrainingCausalLM(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads):
        super().__init__()
        self.attn = CausalSelfAttention(d_model, num_heads)
        self.vocab_proj = nn.Linear(d_model, vocab_size)
        self.loss_fn = nn.CrossEntropyLoss(reduction='mean')

    def forward(self, x, targets=None):
        # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
        # targets: target token ids, shape: (batch_size, seq_len)
        h = self.attn(x)
        logits = self.vocab_proj(h)

        loss = None
        if targets is not None:
            shift_logits = logits[..., :-1, :].contiguous()
            shift_targets = targets[..., 1:].contiguous()
            loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1)
    )

        return logits, loss
```

Task Adaptive Pretraining (TAPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \ldots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^{T} P(x_t \mid x_{<t}) \tag{31}$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \tag{32}$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \tag{33}$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \tag{34}$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{35}$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t. The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{36}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{37}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{38}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \tag{39}$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \tag{40}$$

Architecture Flow Diagram

State Transition Table

Property	Before	After
Task vocabulary alignment	Partial	Strong
Corpus size	Millions of docs	Thousands of docs
Training epochs	1–3	10–100 (small corpus)
Downstream label efficiency	Requires more labels	Requires fewer labels
Transfer to other tasks	Good	Reduced (narrow focus)

Table 4: State properties before and after task adaptive pretraining.

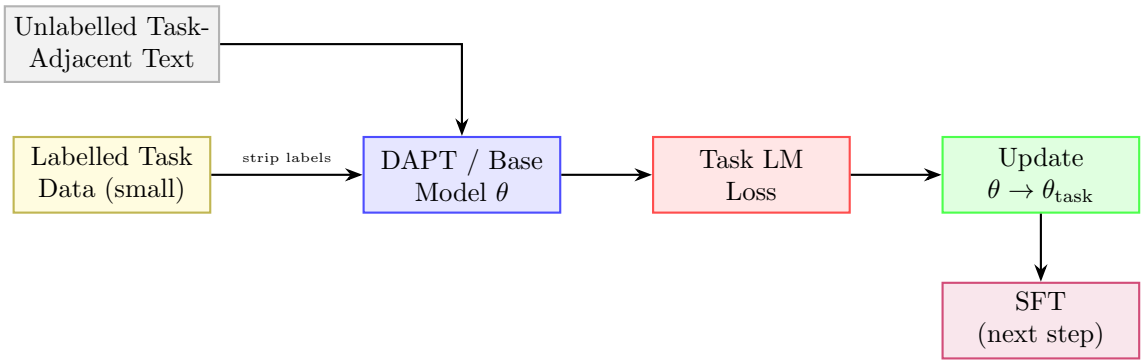

Figure 4: TAPT uses task-adjacent unlabelled text to bridge base model and supervised fine-tuning.

Python Implementation

```python
import torch
import torch.nn as nn
import math

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, x):
        # x shape: (batch_size, seq_len, d_model)
        b, t, c = x.size()

        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)

        # Compute scaled query-key dot product attention
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)

        # Apply causal attention mask
        mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
        scores = scores + mask.view(1, 1, t, t)

        attn_weights = torch.softmax(scores, dim=-1)
        out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
        out = out.transpose(1, 2).contiguous().view(b, t, c)
        return self.out_proj(out)

class PretrainingCausalLM(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads):
        super().__init__()
        self.attn = CausalSelfAttention(d_model, num_heads)
        self.vocab_proj = nn.Linear(d_model, vocab_size)
        self.loss_fn = nn.CrossEntropyLoss(reduction='mean')

    def forward(self, x, targets=None):
        # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
        # targets: target token ids, shape: (batch_size, seq_len)
        h = self.attn(x)
        logits = self.vocab_proj(h)

        loss = None
        if targets is not None:
            shift_logits = logits[..., :-1, :].contiguous()
            shift_targets = targets[..., 1:].contiguous()
            loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1)
    )

        return logits, loss
```

Part II
Supervised Fine-Tuning Methods

Supervised Fine-Tuning (SFT)

Mathematical Foundation

SFT trains the model on structured prompt-response pairs $(x, y) \sim \mathcal{D}_{\text{SFT}}$ using teacher forcing. Let $s = [x; y]$ be the concatenated token sequence of length $M + U$, where $x = (s_1, \ldots, s_M)$ is the prompt and $y = (s_{M+1}, \ldots, s_{M+U})$ is the response. The key math lies in applying a target label mask $L \in \{-100, \mathcal{V}\}^{M+U}$ defined as:

$$L_t = \begin{cases} -100 & \text{if } t \leq M \\ s_t & \text{if } t > M \end{cases} \tag{41}$$

The cross-entropy loss function ignores targets set to -100, directing gradient updates exclusively to the response generation tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{U} \sum_{t=M+1}^{M+U} \log P_\theta(s_t \mid s_{<t}) \tag{42}$$

This loss ignores the prompt distribution $P(x)$, preventing the model from degrading its output generation capability to match arbitrary prompts. The gradients only backpropagate through response tokens:

$$\nabla_\theta \mathcal{L}_{\text{SFT}} = -\frac{1}{U} \sum_{t=M+1}^{M+U} \nabla_\theta \log P_\theta(s_t \mid s_{<t}) \tag{43}$$

Architecture Flow Diagram

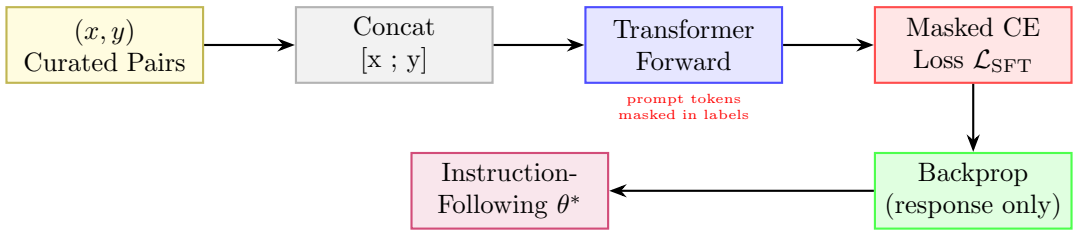

Figure 5: SFT pipeline with prompt-masked loss: gradients flow only through response tokens.

State Transition Table

Property	Before	After
Behaviour	Completion only	Instruction following
Loss region	N/A	Response tokens only
Label masking	None	Prompt masked to -100
Data format	Raw text	(instruction, response) pairs
Output quality	Random completion	Task-aligned responses

Table 5: State properties before and after supervised fine-tuning.

Python Implementation

```python
import torch
import torch.nn as nn

class SFTDatasetCollator:
    def __init__(self, tokenizer, ignore_index=-100):
        self.tokenizer = tokenizer
        self.ignore_index = ignore_index

```

```
    def __call__(self, batch):
        # batch: list of dicts with 'prompt' and 'response' strings
        input_ids_list = []
        labels_list = []

        for item in batch:
            prompt_tokens = self.tokenizer.encode(item['prompt'])
            res_tokens = self.tokenizer.encode(item['response'])

            # Concatenate prompt + response
            input_ids = prompt_tokens + res_tokens
            # Mask the prompt tokens in the label
            labels = [self.ignore_index] * len(prompt_tokens) + res_tokens

            input_ids_list.append(torch.tensor(input_ids))
            labels_list.append(torch.tensor(labels))

        # Pad sequences to max length in batch
        padded_inputs = nn.utils.rnn.pad_sequence(input_ids_list, batch_first=True, padding_value=
    self.tokenizer.pad_token_id)
        padded_labels = nn.utils.rnn.pad_sequence(labels_list, batch_first=True, padding_value=self.
    ignore_index)

        return padded_inputs, padded_labels

class SFTModuleLoss(nn.Module):
    def __init__(self, ignore_index=-100):
        super().__init__()
        self.ce = nn.CrossEntropyLoss(ignore_index=ignore_index)

    def forward(self, logits, targets):
        # Shifting logits and targets for next-token prediction
        shift_logits = logits[..., :-1, :].contiguous()
        shift_targets = targets[..., 1:].contiguous()
        return self.ce(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))
```

Instruction Tuning

Mathematical Foundation

Instruction Tuning exposes the model to multi-turn conversational patterns. A prompt template $T(I, x)$ converts an instruction I and context x into structured sequences. Turn boundaries are defined using chat-formatting systems (like ChatML). A single sequence layout is:

$$S = \langle \text{user} \rangle I \parallel x \langle \text{assistant} \rangle y \langle \text{eos} \rangle \tag{44}$$

Let $\mathcal{D}_{\text{inst}}$ consist of instructions spanning hundreds of tasks. The model parameters θ are optimized on the joint multi-task dataset:

$$\mathcal{L}_{\text{IT}}(\theta) = -\mathbb{E}_{(I,x,y)\sim\mathcal{D}_{\text{inst}}} \sum_{t=1}^{|y|} \log P_\theta\big(y_t \mid T(I,x), y_{<t}\big) \tag{45}$$

We mask all prompt-side tokens, including conversational format tags (like $\langle \text{user} \rangle$ and instruction text), ensuring parameters are trained only on synthesizing assistant responses:

$$\mathcal{L}_{\text{IT}}(\theta) = -\frac{1}{|y|} \sum_{i=1}^{|T(I,x)|+|y|} M_i \log P_\theta(S_i \mid S_{<i}), \quad M_i = \mathbb{I}(i > |T(I,x)|) \tag{46}$$

Architecture Flow Diagram

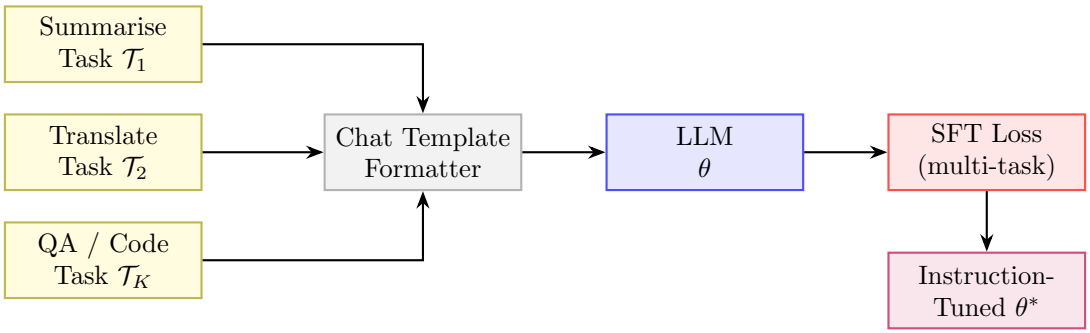

Figure 6: Instruction tuning over diverse task types funnelled through a unified chat template.

State Transition Table

Property	Before	After
Zero-shot generalisation	Poor	Strong across task types
Task diversity in training	N/A	K heterogeneous tasks
Input format	Free-form text	Structured chat template
Prompt sensitivity	High	Reduced
Held-out task performance	Weak	Strong (FLAN, InstructGPT)

Table 6: State properties before and after instruction tuning.

Python Implementation

```python
import torch
import torch.nn as nn

class InstructionFormatter(nn.Module):
    def __init__(self, tokenizer, ignore_index=-100):
        super().__init__()
        self.tokenizer = tokenizer
        self.ignore_index = ignore_index
        self.ce = nn.CrossEntropyLoss(ignore_index=ignore_index)

    def format_chatml(self, instruction, context, response):
```

```
        prompt_text = f"<|im_start|>user\n{instruction}\n{context}<|im_end|>\n<|im_start|>assistant\n"
        response_text = f"{response}<|im_end|>\n"

        prompt_ids = self.tokenizer.encode(prompt_text)
        response_ids = self.tokenizer.encode(response_text)

        input_ids = prompt_ids + response_ids
        targets = [self.ignore_index] * len(prompt_ids) + response_ids

        return torch.tensor(input_ids), torch.tensor(targets)

    def forward(self, logits, targets):
        shift_logits = logits[..., :-1, :].contiguous()
        shift_targets = targets[..., 1:].contiguous()
        return self.ce(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))
```

Multi-Task Training

Mathematical Foundation

Multi-Task Training aggregates losses from K distinct tasks. A simple summation can result in tasks with large gradient scales dominating the parameter updates. To resolve this, we use the GradNorm algorithm. Let W be the shared parameter weights (e.g., the last transformer block weights). We define the gradient norm for task k at training step t as:

$$G_k(t) = \|\nabla_W (w_k(t)\mathcal{L}_k(t))\|_2 \tag{47}$$

Let $\bar{G}(t) = \frac{1}{K}\sum_k G_k(t)$ be the average gradient norm. The target gradient norm for task k balances task difficulties:

$$\hat{G}_k(t) = \bar{G}(t) \times \left(\frac{\tilde{\mathcal{L}}_k(t)}{\frac{1}{K}\sum_j \tilde{\mathcal{L}}_j(t)} \right)^{\alpha} \tag{48}$$

where $\tilde{\mathcal{L}}_k(t) = \mathcal{L}_k(t)/\mathcal{L}_k(0)$ is the task loss ratio indicating task training progress, and α is a hyperparameter managing structural gradient balancing. The task weights $w_k(t)$ are updated by minimizing the L1 distance between $G_k(t)$ and $\hat{G}_k(t)$:

$$\mathcal{L}_{\text{grad}}(w_1, \ldots, w_K; t) = \sum_{k=1}^{K} \left| G_k(t) - \hat{G}_k(t) \right| \tag{49}$$

The final joint parameter objective is:

$$\mathcal{L}_{\text{MT}}(\theta) = \sum_{k=1}^{K} w_k \mathcal{L}_k(\theta) \tag{50}$$

Architecture Flow Diagram

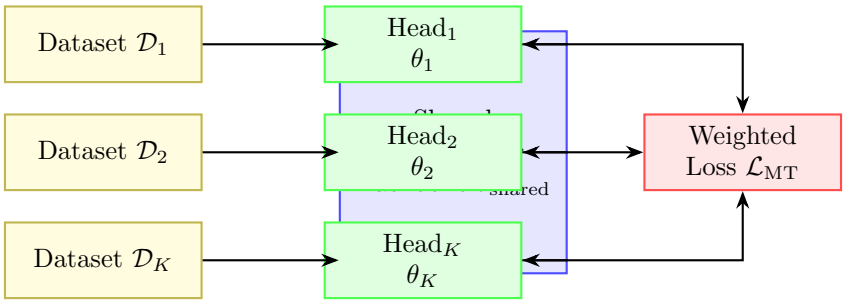

Figure 7: Multi-task training: each dataset feeds task-specific heads on a shared backbone.

State Transition Table

Property	Before	After
Backbone specialisation	Single task	K tasks jointly
Task heads	1	K heads
Generalisation	Narrow	Broad
Data efficiency	Per-task only	Cross-task transfer
Gradient conflicts	N/A	Managed via λ_k

Table 7: State properties before and after multi-task training.

Python Implementation

```python
import torch
import torch.nn as nn

class GradNormMultiTaskTrainer:
    def __init__(self, model, num_tasks, alpha=0.12, lr_w=0.025):
        self.model = model
        self.num_tasks = num_tasks
```

```
        self.alpha = alpha
        self.lr_w = lr_w

        # Learnable task weights, initialized to 1.0
        self.w = nn.Parameter(torch.ones(num_tasks, requires_grad=True))
        self.initial_losses = None
        self.optimizer_w = torch.optim.Adam([self.w], lr=lr_w)

    def train_step(self, task_losses, shared_weights_layer, optimizer_model):
        # task_losses: tensor of shape (num_tasks,) containing current task losses
        if self.initial_losses is None:
            self.initial_losses = task_losses.detach().clone()

        weighted_losses = self.w * task_losses
        total_loss = weighted_losses.sum()

        # Backward pass on the main model parameters
        optimizer_model.zero_grad()
        total_loss.backward(retain_graph=True)

        # Compute gradient norms G_k for each task with respect to shared weights
        norms = []
        for i in range(self.num_tasks):
            grads = torch.autograd.grad(weighted_losses[i], list(shared_weights_layer.parameters()),
 retain_graph=True, create_graph=True)
            # Accumulate squared norms of all parameters in shared weights layer
            norm = torch.sqrt(sum(g.pow(2).sum() for g in grads if g is not None))
            norms.append(norm)
        norms = torch.stack(norms)

        # Calculate loss ratios and target gradient norms
        loss_ratios = task_losses.detach() / self.initial_losses
        mean_norm = norms.mean().detach()
        mean_ratio = loss_ratios.mean()

        targets = mean_norm * (loss_ratios / mean_ratio) ** self.alpha

        # Update task weights w
        self.optimizer_w.zero_grad()
        grad_loss = torch.sum(torch.abs(norms - targets))
        grad_loss.backward()

        # Update weights and normalize them
        self.optimizer_w.step()
        with torch.no_grad():
            self.w.copy_(self.w / self.w.sum() * self.num_tasks)

        optimizer_model.step()
        return total_loss.item(), self.w.detach().tolist()
```

Chain-of-Thought SFT

Mathematical Foundation

CoT SFT trains models to output intermediate reasoning steps before generating the final answer. Let x be the prompt, $c = (c_1, \ldots, c_M)$ be the reasoning chain tokens, and $y = (y_1, \ldots, y_U)$ be the final answer tokens. The joint probability factorization is:

$$P(c, y \mid x) = \prod_{i=1}^{M} P_\theta(c_i \mid x, c_{<i}) \prod_{j=1}^{U} P_\theta(y_j \mid x, c, y_{<j}) \tag{51}$$

The model is trained by minimizing the joint loss:

$$\mathcal{L}_{\text{CoT}}(\theta) = -\beta \sum_{i=1}^{M} \log P_\theta(c_i \mid x, c_{<i}) - (1-\beta) \sum_{j=1}^{U} \log P_\theta(y_j \mid x, c, y_{<j}) \tag{52}$$

During autoregressive generation, we sample tokens using Temperature-scaled softmax and Nucleus (top-p) filtering. The logits z_t are modified as:

$$\tilde{z}_{t,i} = \frac{z_{t,i}}{T} \tag{53}$$

where $T > 0$ is the temperature. The top-p vocabulary subset $\mathcal{V}^{(p)}$ at step t is the smallest set satisfying:

$$\sum_{v \in \mathcal{V}^{(p)}} P(v \mid x, s_{<t}) \geq p \tag{54}$$

Tokens outside $\mathcal{V}^{(p)}$ are assigned probability 0 before final selection.

Architecture Flow Diagram

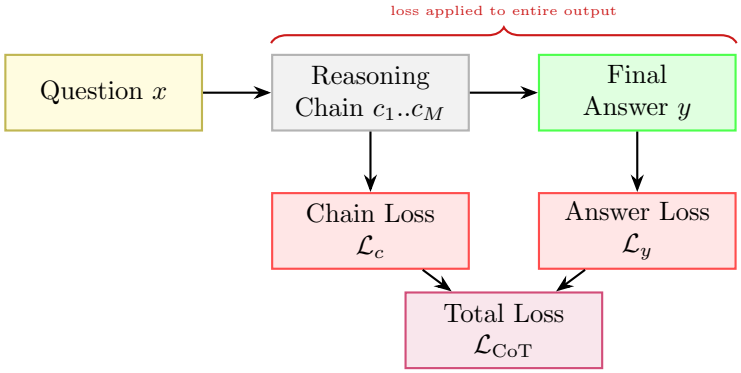

Figure 8: CoT SFT: loss is computed over both the reasoning chain and the final answer.

State Transition Table

Property	Before	After
Output format	Direct answer	Reasoning chain + answer
Training signal per example	$\lvert y \rvert$ tokens	$\lvert c \rvert + \lvert y \rvert$ tokens
Multi-step reasoning	Weak	Strong
Data annotation cost	Low	High (chain annotation)
Accuracy on complex tasks	Low	Substantially improved

Table 8: State properties before and after Chain-of-Thought SFT.

Python Implementation

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class CoTGenerator(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def sample_top_p(self, logits, top_p=0.9, temperature=0.7):
        # logits shape: (1, vocab_size)
        scaled_logits = logits / temperature
        probs = F.softmax(scaled_logits, dim=-1)

        sorted_probs, sorted_indices = torch.sort(probs, descending=True, dim=-1)
        cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

        # Remove tokens with cumulative probability above top_p (but keep the first one)
        sorted_indices_to_remove = cumulative_probs > top_p
        sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
        sorted_indices_to_remove[..., 0] = False

        # Set filtered logits to negative infinity
        indices_to_remove = sorted_indices_to_remove.scatter(1, sorted_indices,
    sorted_indices_to_remove)
        scaled_logits[indices_to_remove] = -1e9

        # Re-compute probabilities and sample
        filtered_probs = F.softmax(scaled_logits, dim=-1)
        next_token = torch.multinomial(filtered_probs, num_samples=1)
        return next_token

    def generate(self, input_ids, max_len=100, top_p=0.9, temp=0.7):
        # Autoregressive generation loop
        generated = input_ids.clone()
        for _ in range(max_len):
            outputs, _ = self.model(generated)
            next_logit = outputs[:, -1, :]
            next_token = self.sample_top_p(next_logit, top_p=top_p, temperature=temp)
            generated = torch.cat([generated, next_token], dim=1)
        return generated
```

Step-by-Step SFT

Mathematical Foundation

Step-by-Step SFT formats the reasoning chain into discrete steps separated by a boundary token $T_{\text{step}} = \texttt{[STEP]}$. Let the response contain S steps: $s = (s_1, T_{\text{step}}, s_2, T_{\text{step}}, \ldots, s_S, T_{\text{step}})$. The step-level loss is:

$$\mathcal{L}_{\text{SBS}}(\theta) = -\sum_{k=1}^{S}\sum_{t=1}^{|s_k|} \log P_\theta(s_{k,t} \mid x, s_{<k}, s_{k,<t}) - \sum_{k=1}^{S} \log P_\theta(T_{\text{step}} \mid x, s_{\leq k}) \tag{55}$$

We define the average step perplexity PPL_k as:

$$\text{PPL}_k = \exp\left(-\frac{1}{|s_k|}\sum_{t=1}^{|s_k|} \log P_\theta(s_{k,t} \mid x, s_{<k}, s_{k,<t})\right) \tag{56}$$

Steps with high perplexity ($\text{PPL}_k > \tau$, where τ is a threshold) indicate low model confidence and can be flagged for step-level revision during generation. The model is trained to minimize the combined cross-entropy loss over all steps.

Architecture Flow Diagram

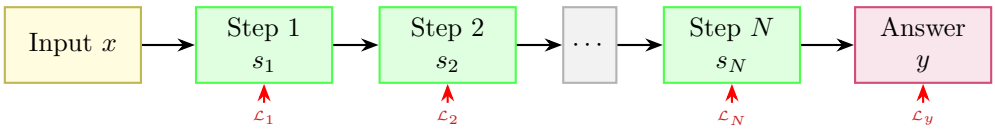

Figure 9: Step-by-Step SFT: each numbered step and the final answer contribute to the total loss.

State Transition Table

Property	Before	After
Solution structure	Unstructured	Numbered procedural steps
Step delimiter tokens	Not in vocabulary	Learned as first-class tokens
Interpretability	Low	Step-level traceability
Annotation format	Raw answer	$(x, [s_1, ..., s_N], y)$
Task decomposition ability	Implicit	Explicit

Table 9: State properties before and after Step-by-Step SFT.

Python Implementation

```python
import torch
import torch.nn as nn

class StepByStepLossAndConfidence(nn.Module):
    def __init__(self, step_token_id=50257, ignore_index=-100):
        super().__init__()
        self.step_token_id = step_token_id
        self.ignore_index = ignore_index
        self.ce = nn.CrossEntropyLoss(reduction='none')

    def forward(self, logits, targets):
        # logits: (batch_size, seq_len, vocab_size)
        # targets: (batch_size, seq_len)
        shift_logits = logits[..., :-1, :].contiguous()
        shift_targets = targets[..., 1:].contiguous()

        batch_size, seq_len = shift_targets.size()

        flat_logits = shift_logits.view(-1, shift_logits.size(-1))
        flat_targets = shift_targets.view(-1)

        # Calculate loss per token position
```

```
        losses = self.ce(flat_logits, flat_targets).view(batch_size, seq_len)

        step_perplexities = []
        for b in range(batch_size):
            step_indices = (shift_targets[b] == self.step_token_id).nonzero(as_tuple=True)[0]
            start = 0
            for idx in step_indices:
                step_loss = losses[b, start:idx.item()]
                # Exclude masked tokens
                valid_losses = step_loss[shift_targets[b, start:idx.item()] != self.ignore_index]
                if len(valid_losses) > 0:
                    step_ppl = torch.exp(valid_losses.mean())
                    step_perplexities.append(step_ppl.item())
                start = idx.item() + 1

        mean_loss = losses[shift_targets != self.ignore_index].mean()
        return mean_loss, step_perplexities
```

Process Supervision

Mathematical Foundation

Process Supervision trains a Process Reward Model (PRM) using step-level binary correctness labels. Let x be the problem description, and $s_1, \ldots, s_S$ be the generated steps. Human or model-based annotations assign correctness labels $r_k \in \{0, 1\}$ for each step s_k. The PRM model outputs correctness probabilities by applying a logistic sigmoid function to the step boundary token representation:

$$\hat{r}_k = \sigma\left(\mathrm{PRM}_\phi(h_{t_k})\right) = \frac{1}{1 + \exp\left(-\mathrm{PRM}_\phi(h_{t_k})\right)} \tag{57}$$

where h_{t_k} is the transformer hidden output at index t_k representing the step boundary token. The PRM parameters ϕ are optimized using binary cross entropy:

$$\mathcal{L}_{\mathrm{PRM}}(\phi) = -\sum_{k=1}^{S} \left[r_k \log \hat{r}_k + (1 - r_k) \log(1 - \hat{r}_k)\right] \tag{58}$$

During inference, candidate sequences are generated. The total sequence correctness score is calculated as the product of step correctness probabilities:

$$\mathrm{Score}(s_{\leq S}) = \prod_{k=1}^{S} \hat{r}_k \tag{59}$$

This score is used to rank candidate rollouts in Best-of-N sampling or guide selection during Monte Carlo Tree Search (MCTS).

Architecture Flow Diagram

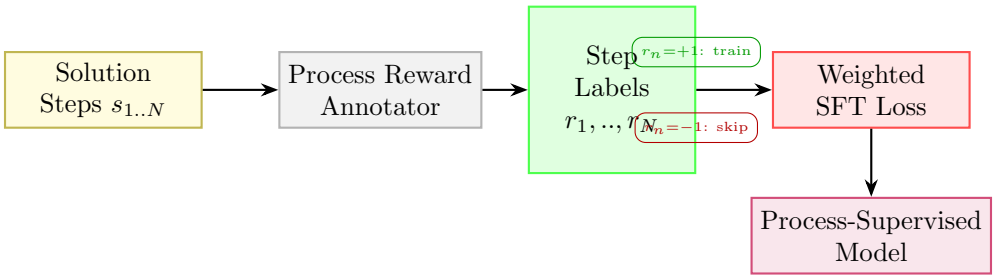

Figure 10: Process supervision: a step annotator labels each reasoning step; only correct steps contribute to the SFT loss.

State Transition Table

Property	Before	After
Supervision granularity	Final answer only	Per-step labels
Error signal	Outcome-level	Step-level
Error localisation	None	Identifies which step failed
Annotation cost	Low	High (step-level labelling)
Reasoning quality	Outcome-driven	Step-correctness driven

Table 10: State properties before and after process supervision.

Python Implementation

```python
import torch
import torch.nn as nn

class ProcessRewardModel(nn.Module):
    def __init__(self, hidden_dim, step_token_id=50257):
        super().__init__()
        self.step_token_id = step_token_id
```

```
        # Binary classification head
        self.value_head = nn.Linear(hidden_dim, 1)
        self.bce = nn.BCEWithLogitsLoss()

    def forward(self, hidden_states, input_ids, step_labels=None):
        # hidden_states shape: (batch_size, seq_len, hidden_dim)
        # input_ids shape: (batch_size, seq_len)
        # step_labels: tensor containing float correctness labels {0.0, 1.0} at step boundaries,
    shape: (batch_size, seq_len)
        logits = self.value_head(hidden_states).squeeze(-1) # (batch_size, seq_len)

        step_mask = (input_ids == self.step_token_id)

        loss = None
        if step_labels is not None:
            # Mask out non-step boundary token positions in loss calculation
            valid_logits = logits[step_mask]
            valid_labels = step_labels[step_mask]
            loss = self.bce(valid_logits, valid_labels)

        # Compute predicted step probabilities
        probs = torch.sigmoid(logits) * step_mask.float()
        return probs, loss
```

Part III
Parameter-Efficient Fine-Tuning (PEFT)

Low-Rank Adaptation (LoRA)

Mathematical Foundation

LoRA represents the weight update matrix $\Delta W \in \mathbb{R}^{k\times d}$ for a linear projection layer as the product of two low-rank matrices $B \in \mathbb{R}^{k\times r}$ and $A \in \mathbb{R}^{r\times d}$, where the rank satisfies $r \ll \min(d, k)$:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx \tag{60}$$

where α is a constant scaling parameter. The base weight matrix $W_0 \in \mathbb{R}^{k\times d}$ is frozen; only B and A are updated. Under a loss $\mathcal{L}$, the gradient propagation rules using the chain rule are derived as follows: Let $z = Ax \in \mathbb{R}^r$ be the intermediate bottleneck activation. Then $h = W_0 x + \frac{\alpha}{r} Bz$. The gradient with respect to output activation is $\nabla_h \mathcal{L} \in \mathbb{R}^{k\times 1}$ (represented as a column vector). The gradient with respect to the intermediate bottleneck representation z:

$$\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} \in \mathbb{R}^{r\times 1} \tag{61}$$

The gradient with respect to the matrix B:

$$\nabla_B \mathcal{L} = \frac{\alpha}{r} \nabla_h \mathcal{L} z^\top = \frac{\alpha}{r} \nabla_h \mathcal{L} x^\top A^\top \in \mathbb{R}^{k\times r} \tag{62}$$

The gradient with respect to the matrix A:

$$\nabla_A \mathcal{L} = \nabla_z \mathcal{L} x^\top = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} x^\top \in \mathbb{R}^{r\times d} \tag{63}$$

To eliminate latency at deployment, the weights are merged directly into the base model:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA \tag{64}$$

Architecture Flow Diagram

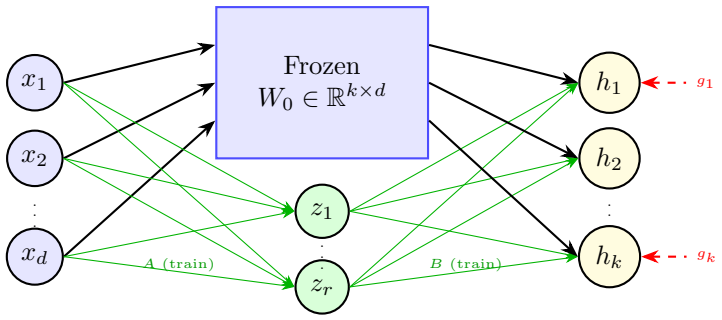

Figure 11: LoRA: input flows through frozen W_0 and trainable low-rank path $A \to B$; outputs are summed.

State Transition Table

Property	Before	After
Base weights W_0	requires_grad = True	Frozen (False)
Trainable params	$k \times d$	$r(k + d) \ll kd$
Matrix B	Does not exist	$\mathbb{R}^{k\times r}$, init $= 0$
Matrix A	Does not exist	$\mathbb{R}^{r\times d}$, init $\sim \mathcal{N}$
Inference overhead	None	Extra BA multiply (or merged)

Table 11: State properties before and after LoRA injection.

Python Implementation

```python
import torch
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=8, alpha=16):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features)
        self.linear.weight.requires_grad = False  # Freeze base weight

        self.r = r
        self.alpha = alpha
        self.scaling = alpha / r

        # Low-rank adapter matrices
        self.lora_A = nn.Parameter(torch.zeros(r, in_features))
        self.lora_B = nn.Parameter(torch.zeros(out_features, r))

        # Initialization
        nn.init.kaiming_uniform_(self.lora_A, a=5**0.5)
        nn.init.zeros_(self.lora_B) # B is initialized to 0, making Delta W = 0 initially

    def merge_weights(self):
        # Update base weights directly
        with torch.no_grad():
            self.linear.weight.add_(self.scaling * (self.lora_B @ self.lora_A))

    def unmerge_weights(self):
        # Subtract back the low rank product
        with torch.no_grad():
            self.linear.weight.sub_(self.scaling * (self.lora_B @ self.lora_A))

    def forward(self, x):
        # Compute forward pass: W0*x + (alpha/r)*B*A*x
        base_out = self.linear(x)
        lora_out = (x @ self.lora_A.t() @ self.lora_B.t()) * self.scaling
        return base_out + lora_out
```

Quantized LoRA (QLoRA)

Mathematical Foundation

QLoRA quantizes the base weight W_0 to 4-bit NormalFloat (NF4) and uses Double Quantization (DQ) to minimize memory requirements. NF4 divides the standard normal distribution $\mathcal{N}(0,1)$ into $2^k = 16$ equal-area intervals. The quantization levels q_i are calculated using the standard normal quantile function Q_X:

$$q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k}\right) + Q_X\left(\frac{i+1}{2^k}\right)\right) \tag{65}$$

The base weights are normalized to $[-1,1]$ using block-wise scaling. Let the block size be $B = 64$. For a block W_b, the scale is $c_b = \max(|W_b|)$. The quantized values are:

$$q_t = \arg\min_{i\in\{0,\ldots,15\}} \left|\frac{W_{b,t}}{c_b} - q_i\right| \tag{66}$$

Double Quantization (DQ) quantizes the scaling factors c_b themselves to 8-bit floats (c_b^{FP8}) with a second scale block size of 256, reducing scale memory overhead from $32/64 = 0.5$ bits/weight to $8/64 + 32/(64 \times 256) \approx 0.127$ bits/weight. During backpropagation, the NF4 weights are dequantized to FP16/BF16 to compute outputs, and gradients are computed only for the low-rank parameters:

$$h = \text{dequant}(c_b, W^{\text{NF4}})x + \frac{\alpha}{r}BAx \tag{67}$$

Architecture Flow Diagram

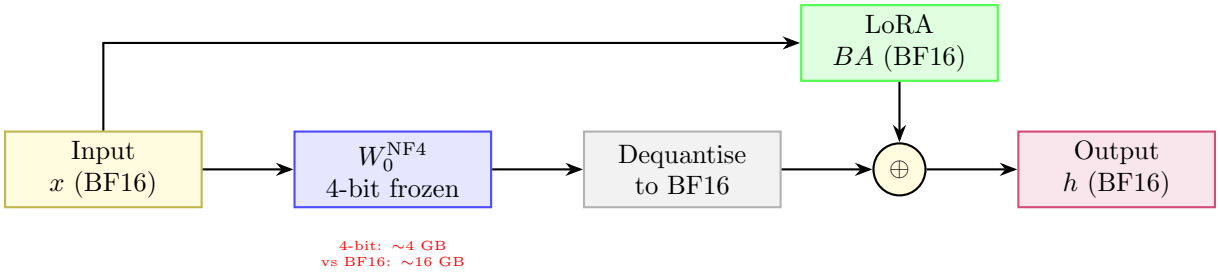

Figure 12: QLoRA: 4-bit frozen base dequantised at runtime; LoRA adapters trained in BF16.

State Transition Table

Property	Before	After
W_0 precision	BF16 / FP16	NF4 (4-bit)
GPU memory for 70B model	~140 GB	~35 GB
LoRA adapter precision	N/A	BF16
Training fidelity	Full	Near-full (straight-through)
Inference latency	Baseline	+dequant overhead

Table 12: State properties before and after QLoRA.

Python Implementation

```python
import torch
import torch.nn as nn

class QLoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=8, alpha=16, block_size=64):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.block_size = block_size
        self.scaling = alpha / r

```

```
        # Simulated NF4 weights stored as half precision, frozen
        self.weight_nf4 = nn.Parameter(torch.randn(out_features, in_features).half(), requires_grad=
    False)
        # Block-wise scales for dequantization
        self.scales = nn.Parameter(torch.zeros(out_features, (in_features + block_size - 1) //
    block_size).half(), requires_grad=False)
        self.compute_simulated_scales()

        self.lora_A = nn.Parameter(torch.zeros(r, in_features))
        self.lora_B = nn.Parameter(torch.zeros(out_features, r))
        nn.init.kaiming_uniform_(self.lora_A, a=5**0.5)
        nn.init.zeros_(self.lora_B)

    def compute_simulated_scales(self):
        # Group weights into blocks and compute max values
        w = self.weight_nf4.data
        for i in range(self.scales.size(1)):
            start = i * self.block_size
            end = min(start + self.block_size, self.in_features)
            block = w[:, start:end]
            self.scales[:, i] = torch.max(torch.abs(block), dim=-1).values

    def dequantize(self):
        # Dequantize simulated NF4 block-by-block
        dequantized = torch.zeros_like(self.weight_nf4)
        for i in range(self.scales.size(1)):
            start = i * self.block_size
            end = min(start + self.block_size, self.in_features)
            scale = self.scales[:, i].unsqueeze(-1)
            dequantized[:, start:end] = self.weight_nf4[:, start:end] * scale
        return dequantized

    def forward(self, x):
        # Dequantize base weights on the fly
        weight = self.dequantize()
        base_out = torch.nn.functional.linear(x, weight)
        lora_out = (x @ self.lora_A.t() @ self.lora_B.t()) * self.scaling
        return base_out + lora_out
```

Adaptive LoRA (AdaLoRA)

Mathematical Foundation

AdaLoRA parameterizes incremental weight updates using a singular value decomposition (SVD) representation to dynamically adjust the adapter rank during training:

$$\Delta W = P \Lambda Q \tag{68}$$

where $P \in \mathbb{R}^{d \times r}$ and $Q \in \mathbb{R}^{r \times k}$ are left and right singular vectors, and $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_r) \in \mathbb{R}^{r \times r}$ contains singular values. To prevent SVD divergence and maintain parameter stability, we enforce orthogonality constraints:

$$\mathcal{L}_{\text{orth}}(P, Q) = \gamma \left(\|P^\top P - I\|_F^2 + \|QQ^\top - I\|_F^2 \right) \tag{69}$$

We estimate the training importance of each singular triplet $(P_{*,i}, \lambda_i, Q_{i,*})$ using gradient-weight product magnitude:

$$S_i = s(\lambda_i) + \frac{1}{d} \sum_{j=1}^{d} s(P_{j,i}) + \frac{1}{k} \sum_{j=1}^{k} s(Q_{i,j}) \tag{70}$$

where $s(\theta) = |\theta \cdot \nabla_\theta \mathcal{L}|$. At designated intervals, triplets with the lowest importance scores S_i are pruned by zeroing out their singular values λ_i, dynamically reducing the adapter rank budget.

Architecture Flow Diagram

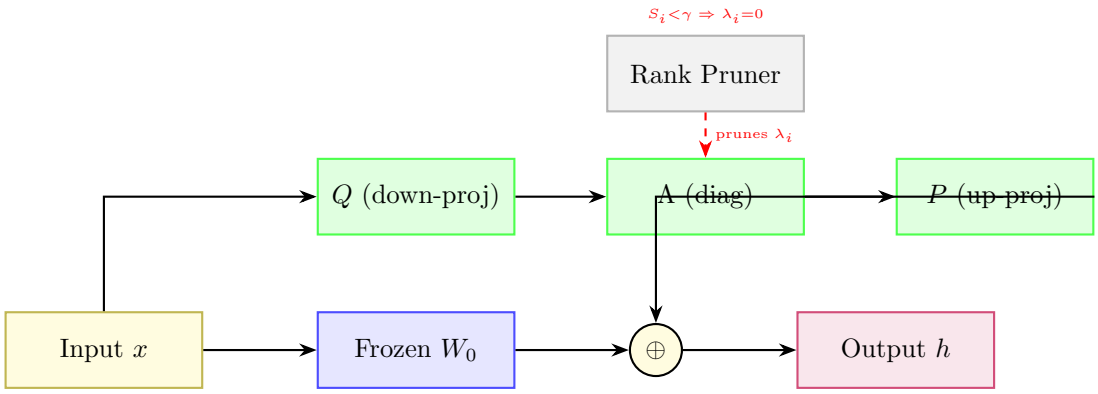

Figure 13: AdaLoRA: SVD-decomposed update with importance-based rank pruning per layer.

State Transition Table

Property	Before	After
Rank per layer	Fixed r	Adaptive (varies by importance)
Update structure	BA product	$P\Lambda Q$ SVD triplet
Parameter budget	Fixed	Redistributed across layers
Low-importance ranks	Included	Pruned ($\lambda_i = 0$)
Training stability	Standard	Constrained by SVD orthogonality

Table 13: State properties before and after AdaLoRA.

Python Implementation

```python
import torch
import torch.nn as nn

class AdaLoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=8, gamma=0.1):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features)
        self.linear.weight.requires_grad = False
        self.r = r
```

```
        self.gamma = gamma

        # SVD Parameterization
        self.lora_P = nn.Parameter(torch.randn(out_features, r))
        self.lora_E = nn.Parameter(torch.randn(r)) # Singular values
        self.lora_Q = nn.Parameter(torch.randn(r, in_features))

        nn.init.orthogonal_(self.lora_P)
        nn.init.zeros_(self.lora_E)
        nn.init.orthogonal_(self.lora_Q)

    def compute_orthogonality_loss(self):
        # P^T * P - I
        p_orth = torch.matmul(self.lora_P.t(), self.lora_P) - torch.eye(self.r, device=self.lora_P.
    device)
        # Q * Q^T - I
        q_orth = torch.matmul(self.lora_Q, self.lora_Q.t()) - torch.eye(self.r, device=self.lora_Q.
    device)
        return self.gamma * (torch.norm(p_orth, p='fro')**2 + torch.norm(q_orth, p='fro')**2)

    def prune_rank(self, keep_mask):
        # keep_mask: boolean tensor of shape (r,) indicating active singular values
        with torch.no_grad():
            self.lora_E.copy_(self.lora_E * keep_mask.float())

    def forward(self, x):
        base_out = self.linear(x)
        # Delta W = P * diag(E) * Q
        delta_W = self.lora_P @ torch.diag(self.lora_E) @ self.lora_Q
        lora_out = x @ delta_W.t()
        return base_out + lora_out
```

Weight-Decomposed LoRA (DoRA)

Mathematical Foundation

DoRA decomposes the weight matrix $W \in \mathbb{R}^{k \times d}$ into its magnitude vector $m \in \mathbb{R}^k$ and direction matrix $V \in \mathbb{R}^{k \times d}$:

$$W = m \cdot \frac{V}{\|V\|_r} = m \cdot \frac{W_0 + \Delta W}{\|W_0 + \Delta W\|_r} \tag{71}$$

where $\| \cdot \|_r$ is the L2 norm computed along each row (corresponding to output channels). In parameter-efficient training, we freeze the direction matrix to the base weight $V = W_0$ and introduce low-rank directional updates using adapters $\Delta V = \frac{\alpha}{r} BA$ (where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$):

$$W = m \cdot \frac{W_0 + \frac{\alpha}{r} BA}{\|W_0 + \frac{\alpha}{r} BA\|_r} \tag{72}$$

Only the magnitude vector m and low-rank directional matrices B and A are trainable. The gradient flow for the directional components is:

$$\nabla_V \mathcal{L} = \frac{m}{\|V\|_c} \left(\nabla_W \mathcal{L} - \frac{V}{\|V\|_c^2} \odot \left(V^\top \nabla_W \mathcal{L} \right) \right) \tag{73}$$

This mathematically separates the updates to magnitude and direction, mimicking full parameter fine-tuning behavior while retaining low parameter overhead.

Architecture Flow Diagram

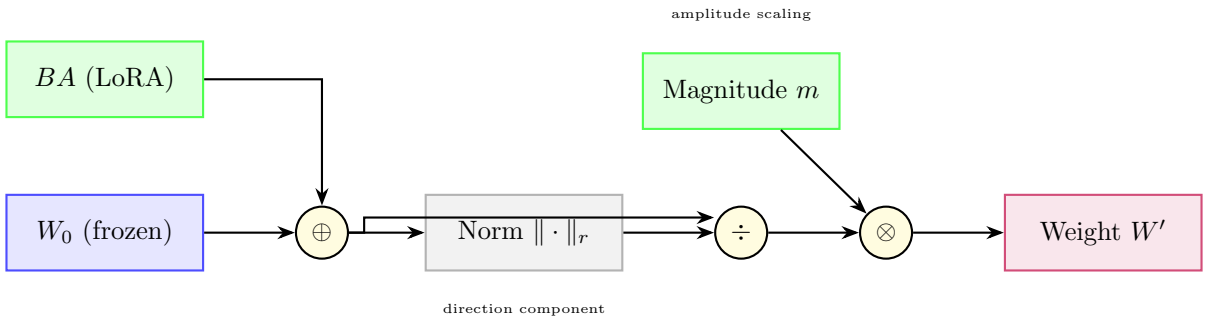

Figure 14: DoRA: weight decomposed into direction (LoRA-adapted) and magnitude (trainable scalar).

State Transition Table

Property	Before (LoRA)	After (DoRA)
Magnitude component	Implicit in W_0	Explicit trainable m
Direction component	$W_0 + BA$ unnorm	Column-normalised
Full FT similarity	Lower	Higher
Trainable params	$r(d+k)$	$r(d+k)+k$
Catastrophic forgetting	Low	Lower

Table 14: State properties comparing LoRA and DoRA.

Python Implementation

```python
import torch
import torch.nn as nn

class DoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=8, alpha=16):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features)
        self.linear.weight.requires_grad = False

        self.r = r
```

```
        self.alpha = alpha
        self.scaling = alpha / r

        # Directional adapter matrices
        self.lora_A = nn.Parameter(torch.zeros(r, in_features))
        self.lora_B = nn.Parameter(torch.zeros(out_features, r))
        nn.init.kaiming_uniform_(self.lora_A, a=5**0.5)
        nn.init.zeros_(self.lora_B)

        # Magnitude parameters m, initialized to column norm of base weights
        W0 = self.linear.weight.data
        self.m = nn.Parameter(torch.norm(W0, p=2, dim=1))

    def forward(self, x):
        W0 = self.linear.weight
        delta_W = (self.lora_B @ self.lora_A) * self.scaling

        # Calculate updated direction matrix V
        V = W0 + delta_W
        V_norm = torch.norm(V, p=2, dim=1, keepdim=True)

        # Construct weight matrix with separate magnitude and direction components
        W = self.m.unsqueeze(1) * (V / V_norm)
        return torch.nn.functional.linear(x, W, self.linear.bias)
```

IA^3 (Infused Adapter by Inhibiting and Amplifying Inner Activations)

Mathematical Foundation

IA^3 introduces element-wise scaling vectors to directly modify inner activations in attention keys, values, and intermediate feed-forward layers. Let the scaling vectors be $l_K \in \mathbb{R}^d$, $l_V \in \mathbb{R}^d$, and $l_{FFN} \in \mathbb{R}^{d_{FFN}}$. The attention layer activations are computed as:

$$\tilde{K} = K \odot l_K = K \cdot \text{diag}(l_K) \tag{74}$$

$$\tilde{V} = V \odot l_V = V \cdot \text{diag}(l_V) \tag{75}$$

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q\tilde{K}^\top}{\sqrt{d_k}}\right)\tilde{V} \tag{76}$$

In the feed-forward network (FFN), let W_1 and W_2 be the weight matrices. The intermediate activations are scaled before the output projection:

$$\text{FFN}(x) = \sigma\big(xW_1 \odot l_{FFN}\big)W_2 = \sigma\big(xW_1 \cdot \text{diag}(l_{FFN})\big)W_2 \tag{77}$$

The base model weights W_Q, W_K, W_V, W_1, W_2 are frozen. This diagonal matrix transformation scales the columns of the projection matrices with minimal parameter updates.

Architecture Flow Diagram

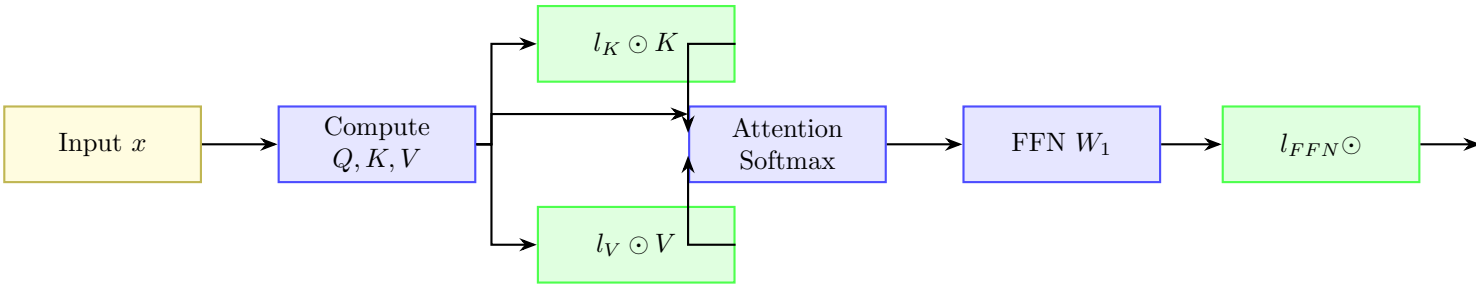

Figure 15: IA^3: three tiny scaling vectors rescale keys, values, and FFN intermediate activations.

State Transition Table

Property	Before	After
Trainable params	0 (frozen)	$3dL$ scalars
Init value of l vectors	N/A	**1** (identity behaviour)
K, V, FFN activations	Unchanged	Element-wise scaled
Param overhead vs LoRA	N/A	$\sim$10$\times$ fewer params
Task adaptation strength	None	Moderate

Table 15: State properties before and after IA^3 injection.

Python Implementation

```python
import torch
import torch.nn as nn

class IA3AttentionWrapper(nn.Module):
    def __init__(self, key_dim, value_dim):
        super().__init__()
        # Scaling vectors initialized to ones
        self.l_K = nn.Parameter(torch.ones(key_dim))
        self.l_V = nn.Parameter(torch.ones(value_dim))

    def scale_keys(self, K):
        # K shape: (batch_size, num_heads, seq_len, head_dim) or (batch_size, seq_len, key_dim)
        return K * self.l_K
```

```
14
15      def scale_values(self, V):
16          return V * self.l_V
17
18  class IA3FFNWrapper(nn.Module):
19      def __init__(self, ffn_dim):
20          super().__init__()
21          self.l_FFN = nn.Parameter(torch.ones(ffn_dim))
22
23      def scale_hidden(self, hidden_states):
24          # hidden_states: intermediate feedforward layer activations
25          return hidden_states * self.l_FFN
```

Adapter Tuning

Mathematical Foundation

Adapter tuning inserts small bottleneck layers into the transformer block, typically after attention and feed-forward modules. Let $h \in \mathbb{R}^d$ be the input hidden state. In the Pfeiffer architecture, a single adapter is inserted per block. In the Houlsby architecture, two adapters are inserted per block. The mathematical transformation for an adapter block is:

$$h' = h + \sigma(hW_{down})W_{up} \tag{78}$$

where $W_{down} \in \mathbb{R}^{d \times r}$ projects to a bottleneck dimension $r \ll d$, σ is an activation function (typically GELU), and $W_{up} \in \mathbb{R}^{r \times d}$ projects back to the original dimension. To ensure the adapter behaves as an identity mapping at the start of training, W_{up} is initialized to zero:

$$h'\big|_{t=0} = h + \sigma(hW_{down}) \cdot 0 = h \tag{79}$$

Architecture Flow Diagram

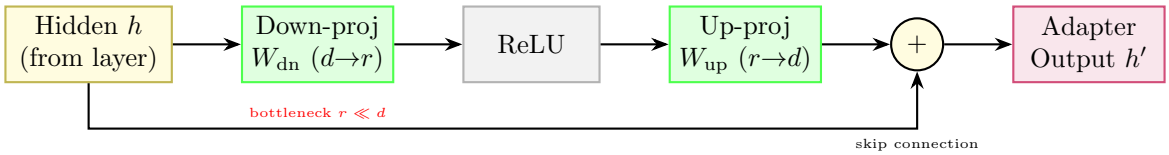

Figure 16: Adapter module: down-project to rank r, apply nonlinearity, up-project back, plus skip connection.

State Transition Table

Property	Before	After
Sub-layer output path	Direct pass-through	+adapter bottleneck
Transformer weights	Trainable	Frozen
New params per layer	0	$2dr + 2d$
Adapter init	N/A	Near-identity ($W_{\text{dn}} \sim \mathcal{N}$)
Inference latency	Baseline	+two extra matmuls per layer

Table 16: State properties before and after adapter insertion.

Python Implementation

```python
import torch
import torch.nn as nn

class BottleneckAdapter(nn.Module):
    def __init__(self, d_model, bottleneck_dim=16):
        super().__init__()
        self.down_proj = nn.Linear(d_model, bottleneck_dim)
        self.act = nn.GELU()
        self.up_proj = nn.Linear(bottleneck_dim, d_model)
        self.layer_norm = nn.LayerNorm(d_model)

        # Initialize up projection to zero to act as identity mapping at start
        nn.init.normal_(self.down_proj.weight, std=1e-3)
        nn.init.zeros_(self.up_proj.weight)

    def forward(self, x):
        residual = x
        out = self.down_proj(x)
        out = self.act(out)
        out = self.up_proj(out)
        # Apply layer normalization with residual connection
        return self.layer_norm(residual + out)
```

Prefix Tuning

Mathematical Foundation

Prefix Tuning prepends continuous task-specific virtual prefix vectors $P_K, P_V \in \mathbb{R}^{l \times d}$ directly to the Key and Value matrices at every transformer layer. Let the original Key and Value matrices for sequence length T be $K, V \in \mathbb{R}^{T \times d}$. The modified Key and Value matrices become:

$$\tilde{K} = [P_K; K] \in \mathbb{R}^{(l+T) \times d} \tag{80}$$

$$\tilde{V} = [P_V; V] \in \mathbb{R}^{(l+T) \times d} \tag{81}$$

To stabilize optimization, during training the prefix is reparameterized using a Multilayer Perceptron (MLP) over a small source embedding E_k:

$$P_K = \text{MLP}_K(E_k), \quad P_V = \text{MLP}_V(E_k) \tag{82}$$

where $E_k \in \mathbb{R}^{l \times d_{\text{mid}}}$. The MLP is discarded at inference, and only the generated static prefix matrices P_K, P_V are saved and used. The attention output is computed as:

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q[P_K; K]^\top}{\sqrt{d_k}}\right)[P_V; V] \tag{83}$$

Architecture Flow Diagram

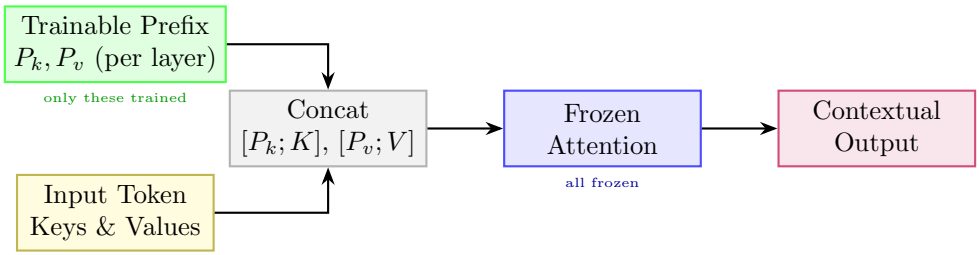

Figure 17: Prefix Tuning: trainable prefix vectors extend the key/value context at every attention layer.

State Transition Table

Property	Before	After
Attention context length	T tokens	$T + l$ tokens (prefix prepended)
Transformer params	Trainable	Fully frozen
Trainable params	0	$2 \times l \times d_h \times L$
Prefix init	N/A	Via MLP reparameterisation
Task switch at inference	Reload model	Swap prefix only

Table 17: State properties before and after prefix tuning.

Python Implementation

```python
import torch
import torch.nn as nn

class PrefixTuningAttention(nn.Module):
    def __init__(self, d_model, num_heads, prefix_len=10):
        super().__init__()
        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads
        self.prefix_len = prefix_len

        # Key/Value projection layers
        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
```

```
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, x, past_keys=None, past_values=None):
        # past_keys, past_values: prefix cache parameters, shape (batch_size, num_heads, prefix_len, head_dim)
        b, t, c = x.size()

        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)

        if past_keys is not None and past_values is not None:
            # Prepend virtual prefix keys/values to the computed sequences
            k = torch.cat([past_keys, k], dim=-2)
            v = torch.cat([past_values, v], dim=-2)

        scores = torch.matmul(q, k.transpose(-2, -1)) / (self.head_dim ** 0.5)
        attn_weights = torch.softmax(scores, dim=-1)
        out = torch.matmul(attn_weights, v)
        out = out.transpose(1, 2).contiguous().view(b, t, c)
        return self.out_proj(out)
```

Prompt Tuning

Mathematical Foundation

Prompt tuning prepends p learnable embedding vectors $P \in \mathbb{R}^{p \times d_{emb}}$ to the input word representations:

$$\tilde{X} = [P; X_e] \in \mathbb{R}^{(p+T) \times d_{emb}} \tag{84}$$

where $X_e \in \mathbb{R}^{T \times d_{emb}}$ is the original sequence word embedding matrix. The entire transformer backbone parameters θ_{base} remain frozen, and gradients are backpropagated only to the prompt parameters P:

$$\nabla_P \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \tilde{X}_{1..p}} \tag{85}$$

During backpropagation, the gradients are passed through all layers of the frozen transformer and update only the prompt embeddings.

Architecture Flow Diagram

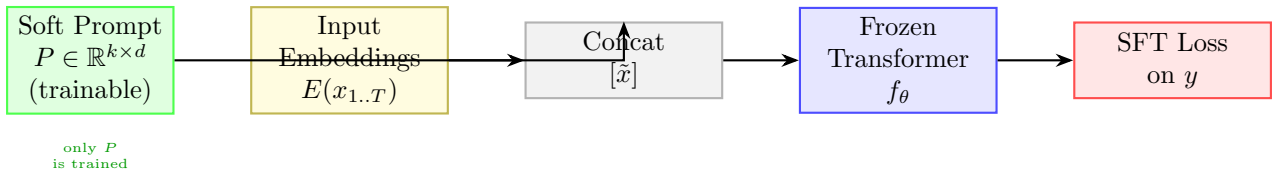

Figure 18: Prompt Tuning: soft tokens prepended at the embedding layer; the full transformer is frozen.

State Transition Table

Property	Before	After
Trainable params	Full model	$k \times d_{\text{emb}}$ only
Prompt format	Discrete hand-crafted	Continuous soft vectors
Transformer weights	All trainable	All frozen
Task switching	Reload fine-tuned model	Swap P only
Sensitivity to init	N/A	Strong (init from class labels helps)

Table 18: State properties before and after prompt tuning.

Python Implementation

```python
import torch
import torch.nn as nn

class PromptTuningEmbedding(nn.Module):
    def __init__(self, base_embeddings, prompt_len=10):
        super().__init__()
        self.base_embeddings = base_embeddings
        for p in self.base_embeddings.parameters():
            p.requires_grad = False

        self.prompt_len = prompt_len
        self.prompt_embeddings = nn.Parameter(torch.randn(prompt_len, base_embeddings.embedding_dim)
    )

    def forward(self, input_ids):
        # input_ids shape: (batch_size, seq_len)
        batch_size = input_ids.size(0)
        base_embs = self.base_embeddings(input_ids) # (batch_size, seq_len, d_emb)

        # Expand prompt embeddings to batch size
        prompts = self.prompt_embeddings.unsqueeze(0).expand(batch_size, -1, -1) # (batch_size,
    prompt_len, d_emb)

        # Concatenate prompt prefix with token embeddings
        return torch.cat([prompts, base_embs], dim=1)
```

P-Tuning v2

Mathematical Foundation

P-Tuning v2 extends prompt tuning by prepending learnable virtual tokens $P^{(l)} \in \mathbb{R}^{p \times 2d_{attn}}$ directly to the attention key and value states at every layer $l \in [1, L]$ of the network. Unlike prefix tuning, P-Tuning v2 removes the MLP reparameterization network, directly optimizing the prefix parameters $P^{(l)}$:

$$\tilde{K}^{(l)} = [P_K^{(l)}; K^{(l)}] \tag{86}$$

$$\tilde{V}^{(l)} = [P_V^{(l)}; V^{(l)}] \tag{87}$$

This deep prefix injection resolves the capacity limitation of shallow prompt tuning, allowing small models (< 10B parameters) to achieve performance comparable to full fine-tuning.

Architecture Flow Diagram

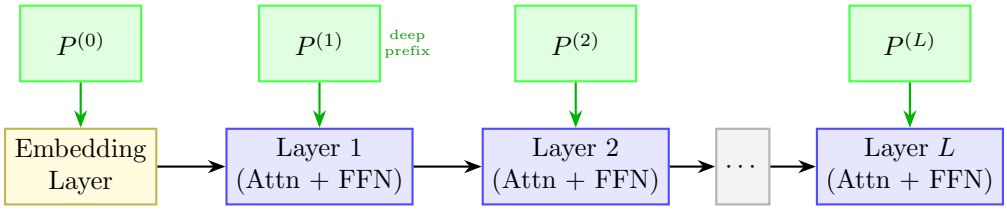

Figure 19: P-Tuning v2: independent trainable prefix tokens injected at every transformer layer.

State Transition Table

Property	Before (Prompt Tuning)	After (P-Tuning v2)
Prefix layers	Embedding only	All L layers
Trainable params	$k \times d$	$L \times l \times d$
NLU performance	Degrades at small scale	Competitive at all scales
Conditioning depth	Shallow (first layer)	Deep (every layer)
Task-specific storage	Single P matrix	L prefix matrices

Table 19: State properties comparing Prompt Tuning and P-Tuning v2.

Python Implementation

```python
import torch
import torch.nn as nn

class PTuningV2Model(nn.Module):
    def __init__(self, num_layers, num_heads, head_dim, prefix_len=10):
        super().__init__()
        self.num_layers = num_layers
        self.num_heads = num_heads
        self.head_dim = head_dim
        self.prefix_len = prefix_len

        # Directly optimized layer-wise prefixes
        self.prefixes = nn.ParameterList([
            nn.Parameter(torch.randn(2, num_heads, prefix_len, head_dim))
            for _ in range(num_layers)
        ])

    def forward(self, batch_size, layer_idx):
        layer_prefixes = self.prefixes[layer_idx]
        return layer_prefixes.unsqueeze(1).expand(-1, batch_size, -1, -1, -1)
```